

C - Port Selector

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

uint16_t* availablePorts(uint16_t *ports, size_t size);
void sort(uint16_t *a, int size);
void printPortList(uint16_t *ports, int count);

int main() {
    uint16_t ports[] = {22, 1024, 1025, 10022, 22, 43, 1082};
    size_t size = sizeof(ports)/sizeof(ports[0]);
    sort(ports, size);
    uint16_t *available = availablePorts(ports, size);

    if (available != NULL) {
        printPortList(available + 1, available[0]);
        free(available);
    }
    return 0;
}

void printPortList(uint16_t *ports, int count) {
    for (int i = 0; i < count; i++) {
        printf("%u\n", ports[i]);
    }
}

void sort(uint16_t *array, int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (array[j] > array[j + 1]) {
                uint16_t temp = array[j];
```

```

        array[j] = array[j + 1];
        array[j + 1] = temp;
    }
}
}

uint16_t* availablePorts(uint16_t *ports, size_t size) {
    size_t count = 0;
    for (size_t i = 0; i < size; i++) {
        if (ports[i] >= 1024 && ports[i] <= 5000) {
            count++;
        }
    }

    if (count > 0) {
        // allocate memory for count + the ports themselves,
        // storing count as the first element
        uint16_t* available = malloc((count + 1) *
sizeof(uint16_t));
        if (available == NULL) {
            fprintf(stderr, "Memory allocation failed\n");
            return NULL;
        }

        available[0] = (uint16_t)count; // store count at the
first position; cast to uint16_t to match type

        int j = 1; // start storing from the second position

        for (int i = 0; i < size; i++) {
            if (ports[i] >= 1024 && ports[i] <= 5000) {
                available[j++] = ports[i];
            }
        }
        return available;
    }
}

```

```
return NULL;
```

```
}
```